Loops in JavaScript

Loops help you repeat code easily. Instead of writing the same lines many times, loops do it for you, making code shorter and faster.

Real-World Example

Think about showing 100 items on a page. Loops let you do this with a few lines, not 100 separate ones.

```
console.log("1");
console.log("2");
console.log("3");
console.log("4");
console.log("5");
```

The For Loop Structure

The `for` loop is very common. It has three main parts: setup, checking, and updating.

```
for(initialisation; condition; updation) {
    //do something
}


for (let i=1; i<=5; i++) {
    console.log(i);
}
```

Initialization

Where the loop begins (e.g., `i = 0`)

Condition

When the loop should stop (e.g., `i < 10`)

Increment

How the loop changes each time (e.g., `i++`)

Print all odd numbers (1 to 15)

Print all even numbers (2 to 10)

Print the multiplication table for 5

Infinite Loops

```
for(let i=1; i>=0; i++) {

}

for(let i=1; i<=5; i--) {

}

for(let i=1; ; i++) {

}
```

Nested for loop

```
for (let i=1; i<=3; i++) {
    for (let j=1; j<=3; j++) {
        console.log(j);
    }
}
```

while loop

```
while (condition) {
  //do something
}
```

```
let i = 1;
while (i <= 5) {
  console.log(i);
  i++;
}
```

Practice Question

Favourite Movie:

```
const favMovie = "avatar";

let guess = prompt("guess my favorite movie");

while( (guess != favMovie) && (guess != "quit")) {
    guess = prompt("wrong guess. please try again");
}

if(guess == favMovie) {
    console.log("congrats!!");
}
```

The Break Keyword

The `break` keyword stops a loop immediately, even if the condition isn't met yet. It's useful when you find what you're looking for and don't need to keep looping.

```
for (let i = 0; i < 10; i++) {
  console.log(`Checking number: ${i}`);
  if (i === 3) {
    console.log("Found 3! Stopping the loop.");
    break; // This stops the loop instantly
  }
}
console.log("Loop ended.");
```

Use `break` when you've achieved your goal within a loop (like finding a specific item) and can stop early to save time and resources.

Loops with Arrays

```
let fruits = ["mango", "apple", "banana", "litchi", "orange"];

for(let i=0; i<fruits.length; i++) {
    console.log(i, fruits[i]);
}
```

```
for(let i=fruits.length-1; i>=0; i--) {
    console.log(i, fruits[i]);
}
```

Nested Loops with Nested Arrays

```
let heroes = [ ["ironman", "spiderman", "thor"], ["superman", "wonder

for(let i=0; i<heroes.length; i++) {
    console.log(`List #${i}`);
    for(let j=0; j<heroes[i].length; j++) {
        console.log(heroes[i][j]);
    }
}
```

for of loop

```
for (element of collection) {

  //do something

}
```

```
let fruits = ["mango", "apple", "banana", "litchi", "orange"]

for(fruit of fruits) {
    console.log(fruit);
}
```

Nested for of loop

```
let heroes = [ ["ironman", "spiderman", "thor"], ["superman", "wonder woman", "flash"]];

for(list of heroes) {
    for(hero of list) {
        console.log(hero);
    }
}
```

Todo App

- list - to show all todos
- add - to add a todo
- delete - to delete a task
- quit - to exit the todo